

## • Easy # 121 → Best Time to Buy & Sell Stock

▲ Problem Statement → Given an array "prices" where "prices[i]" represents the stock price on day "i", find the maximum profit possible by buying once & selling once in the future.

If no profit is possible, return "0".

▲ Main Problem Solution → Traverse the array once while keeping track of:-

- The "minimum price" seen so far.
- The "maximum profit" possible by selling on the current day.

For every price:-

$$\text{profit} = \text{current price} - \text{minimum price}$$

Update the maximum profit whenever we find a better one.

▲ Brute Force Method → Try every possible pair of buying & selling days & calculate the profit.

Time →  $O(n^3)$

Space →  $O(1)$

▲ Algorithm → 1. Initialize "min price" with the first price.

2. Initialize "max profit = 0"

3. Traverse through every price.

4. Update "min price" if the current price is smaller.

5. Calculate the profit by selling at the current price.

6. Update "max profit" if the current profit is larger.

7. Return "max profit".

▲ Important Points → • Keep track of the lowest buying price so far.

• The current day can be considered as the selling day.

• Buy always happens before sell because "min price" comes from an earlier / current position.

• If prices keep decreasing, "max profit" remains "0".

• Only one pass through the array is ~~that~~ needed.

▲ Complexity → Time →  $O(n)$   
Space →  $O(1)$